PBL MILESTONE 1 · ALGOCOM × SOFTENG · REVISION 2

Project LIBRAS

Library Borrowing and Records Administration System: dual-core algorithmic optimization on a decoupled service-layer architecture

Group 1 · III-BCSAD

University of Makati · CCIS · AY 2026–2027

Advisers

Prof. Nomar Maestro · Prof. Era Marie Gannaban

THE PROBLEM

Circulation at UMak LLC must meet strict SLAs

<30 s
Checkout (was 4 min 10 s by manual ledger)

<15 s
Routine book return

<1.0 s
Catalog search under 50 concurrent patrons

Two computational bottlenecks

Typo-tolerant search: SQL `ILIKE '%term%'` is a substring match with no typo tolerance. It found only 3.7% of the words within two edits, and it still scans every row.

Hold dispatch: one returned copy needs its top-priority patron resolved instantly, with no race conditions or starvation.

PROBLEM FORMULATION

Two coupled optimization problems

1 · Approximate string match

Minimize $d_{Lev}(Q, w)$ subject to $d \le k_{max} = 2$

Rank by $(1 - d / \max(|Q|,|w|)) \cdot IDF \cdot Popularity$

2 · Priority scheduling

Select $r^* = \text{argmin PriorityScore}(r_i)$

where $PriorityScore = t_i - W(\rho_i)$; smaller means higher priority

Inputs: lexicon D with $N \ge 50{,}000$ tokens · query Q with $1 \le |Q| \le 64$ · hold pool R with $K \le 5{,}000$ requests

CONSTRAINTS

Boundaries and role-tier weights

Hard boundaries

- Query truncated at 64 characters on ingress
- Search radius capped at $k_{max} = 2$
- Arrival timestamps strictly increasing (NTP-synced)
- Claim window of 48 h, then automatic expiry

Role priority shift

Role	Shift
Faculty	+24 h advance
NTP	+12 h advance
Student	0 (pure FIFO)
Visitor	−24 h deprioritized

ALGORITHM SELECTION

Dual-engine architecture

Engine 1 · Bounded Levenshtein Trie Search

Dynamic programming over a prefix tree. Shared prefixes are evaluated once, and branch-and-bound pruning skips dead subtrees. Stored as flat typed arrays.

Engine 2 · Role-Weighted Priority Queue Dispatcher

Greedy choice on an ordered index: the first entry is always the winning patron. Implemented as an indexed PostgreSQL query.

Alternatives tested and not chosen

Search: SymSpell (318 MB), per-node-object trie (55 MB), BK-tree, bigram index, `pg_trgm` (17% recall), `ILIKE` (3.7% recall), linear scan.

Queue: 4-ary, lazy-delete and pairing heaps, sorted array, unsorted scan. Full results on slide 9.

ALGORITHM MECHANICS · SEARCH

DP rows travel down the trie

Recurrence

Match: $D(i,j) = D(i-1,j-1)$

Else: $1 + \min(\text{delete}, \text{insert}, \text{substitute})$

One row of $m + 1$ cells per trie node

Pruning invariant

If $\min(\text{Row}_u) > 2$, no descendant can recover (Lemma 2), so the whole subtree is skipped with zero false negatives.

96.7%

of trie nodes unvisited at $k = 2$ (99.6% at $k = 1$)

Result: recall 100% against a brute-force reference on every benchmark query.

ALGORITHM MECHANICS · DISPATCH

An ordered index keeps the winner first

Enqueue

`INSERT` into `holds`

O(log K)

Peek

`SELECT ... ORDER BY score, seq LIMIT 1`

O(log K)

Dispatch

`DELETE ... FOR UPDATE SKIP LOCKED`, one transaction with the copy status update

O(log K)

Worked example

Faculty arriving at t scores t − 24 h and beats a Student who arrived at t − 12 h, so the returned copy goes to Faculty. With 50 concurrent clients, each of 2,000 holds was served exactly once.

SCALABILITY ANALYSIS

Complexity and resource profile

Operation	Worst case	Measured (synthetic catalog)
Fuzzy search	$O(m^3 \cdot \lvert\Sigma\rvert^3)$ nodes; $O(m^4 \cdot \lvert\Sigma\rvert^3)$ time	About 8,400 nodes at m = 12, k = 2; p95 0.73 ms at 50,000 tokens
Hold enqueue / dispatch	O(log K)	About 1.7 ms through PostgreSQL (limit 10 ms)
Hold peek	O(log K)	Index lookup; in-memory heap is O(1)
Trie memory	$O(N \cdot L_{avg})$	4.4 MB at 50,000 tokens; 16.9 MB at 200,000
Queue memory	O(K) in PostgreSQL	None in the Node.js process

EMPIRICAL SELECTION

Why these algorithms: measured against alternatives

Search candidate (50,000 tokens, k = 2)	p95 (ms)	Memory	Outcome
SymSpell	0.19	318 MB	Fastest; fails memory rule
Trie, object per node	2.93	55 MB	Fails memory rule
Bigram index	1.97	5.6 MB	Passes; slower as catalog grows
Flat trie (chosen)	**1.93**	**4.4 MB**	**Passes; fastest from 50,000 tokens up**

Rule: exact results, at least 250 queries/s, memory within 2× the smallest passing option, then lowest p95. In repeated runs the flat trie p95 was 0.73 ms against 1.25 ms (bigram) at 50,000 tokens and 2.4 ms against 21.6 ms at 400,000.

Queue: every in-memory heap is about 50,000× inside the 10 ms limit; the PostgreSQL query (1.7 ms) was chosen for durability, atomicity with the loan record, and no queue state in the application.

DECOUPLED SERVICE-LAYER ARCHITECTURE

Algorithms are modules behind narrow interfaces

Presentation · React Native borrower app · React.js librarian portal

↓ HTTPS

Hosting and ingress (Render) · HTTPS, JWT, CORS, rate limit 100 req/min in Express middleware

↓ Internal call

Node.js / Express / TypeScript · business rules, mailer · **SearchEngine** (flat trie, atomic swap) · **HoldQueue** (SQL repository)

↓ SQL, row locking

PostgreSQL 16 · catalog, accounts, loans, fines · hold queue with index on (title, score, seq)

BENCHMARKING AND VALIDATION

Empirical validation against the SLAs

DEV-ALGO-101 · Search

- Results equal a brute-force reference for k = 0, 1, 2
- p95 under 35 ms at 50,000 tokens
- Coverage above 95%

DEV-ALGO-102 · Dispatch

- Order equals the reference heap
- Under 10 ms at K = 1,000 and 5,000
- No duplicate serves across 50 concurrent clients

Tooling

Node.js benchmark harness (`process.hrtime`, forced GC) · PostgreSQL 16 · `node:test`

Load target

Search ≤ 1.0 s at 50 users

Method: Big-O before, profiling after

KEY CITATIONS

IEEE-referenced foundations

Algorithm foundations

[1] Levenshtein, 1966

[2] Cormen et al., CLRS 4th ed., 2022

[9] Navarro, ACM Comput. Surv., 2001

Automata and search

[10] Schulz and Mihov, IJDAR, 2002

[23] Ukkonen, Inf. Control, 1985

[24] Burkhard and Keller, CACM, 1973

[25] Gravano et al., VLDB, 2001

Architecture and data

[13] Fowler, PoEAA, 2002

[17] Newman, Building Microservices, 2021

[28] PostgreSQL 16 documentation

Full list [1]–[28] in the proposal · github.com/necesario0/LIBRAS